

COLX 531: Membership Attack Project: Milestone 2

Yucai Zhong and Christina McCallum and Jennifer Flake and Rachelle De Jager

Abstract

Membership inference on clinical texts was performed by using both SmoLM2-135M and SmoLM2-360M language models, to process patient discharge notes. We implemented the following three models to perform Membership Inference Attacks: Model 1: Min-k% and XGBoost, Model 2: Cross-Model Delta Features with Gradient Boosting, and Model 3: Partial SPV-MIA with min-K% and XGBoost. Each model directly or indirectly built on the baseline model developed in Milestone 1. Each of the three models had results on that were an improvement over the results of the baseline model.

1 Introduction

2 Model Development

2.1 Model 1: Min-k% and XGBoost

2.1.1 Description

This model directly builds on the baseline developed in Milestone 1. A few new methods were added to what was developed last week to see if we could boost our results. First, from the papers read last week, it became clear that the metric min-k% may be a useful indicator of membership. It uses k% of tokens with the lowest likelihoods: members, which have appeared in the fine-tuning data, will assign a high likelihood to even the least likely tokens in a sequence. If the least likely tokens still have a high probability, it's indicative that the sample may have been a member of the training set. (Duan et al. 2024). After including cross-model differences and ratios, we tried an XGBoost model in addition to a Logistic Regression and Random Forest from last week, to see if the complex, non-linear interactions between these new features could be better captured.

2.1.2 Implementation

We implemented this method as a Python script in the src folder using Google Colab. We used the same pipeline as last week, extracting features perplexity, loss, min/max/STD/mean token probability, low-confidence token ratio, and top-k mass. We added minimum-k% at k=10, 20, and 30. We then computed cross-model differences and ratio features to capture the relationship between the two models' outputs. In initial experiments, we observed that the 360M model produced more discriminative features, showing a larger separation between members and non-members, so the difference and ratio features capture how much more confident the 360M model is relative to the 135M model.

We also experimented with ReCaLL: Relative Conditional Log Likelihood. This measures whether a target model assigns higher likelihood to a text compared to a reference model, conditioned on non-member prefixes (Xie et al. 2024). However, after running the pipeline through once and examining feature importances, we found that ReCaLL wasn't ranked as important by any of the classifiers, and seemed to be adding random noise (the AUC score decreased compared to the baseline on this initial run). We therefore took this metric out, and were able to score higher with just min-k% and cross-model difference and ratio features.

After this feature extraction was complete, we trained an XGBClassifier model on the training split and evaluated on the validation split using the official metrics below.

2.1.3 Results

Method	AUC	TPR@FPR=0.1
Min-K XGBoost	0.8955	0.6688

Table 1: Validation performance of Model 1.

2.1.4 Discussion

This method achieved a test score of $AUC = 0.90548$ on the kaggle test set. An advantage of this method is that, building on our baseline, perplexity and loss are simple and interpretable but can be unreliable on their own, since inherently easy texts can have low perplexity regardless of their membership/non-membership tag. Our addition of min-k% features addresses this by focusing on harder tokens, where memorized texts still show high confidence. Cross-model difference and ratio features help control for text difficulty by comparing the 360M model’s confidence against the 135M model. Among our classifiers, XGBoost performed the best ($AUC\ 0.8955$ compared to linear regression at 0.8902 and Random Forest at 0.8851), making it clear that it captures the non-linear relationships between features successfully. One downside to our approach is the computational cost of feature extraction: we ran every text through two full models, which was computationally expensive. Further, we could have implemented some hyperparameter tuning, including testing out different values of k for min-k%, and different parameters within XGBoost like learning rate and max depth.

2.2 Model 2: Cross-Model Delta Features with Gradient Boosting

2.2.1 Description

This method extends our milestone 1 baseline by introducing additional features. In the baseline, we relied on confidence-related features from the 135M and 360M models, such as loss, perplexity, and token probability statistics. In this second method, we extract the same feature family from the same two models and then construct cross-model delta features to measure how differently the two models behave on the same input. This makes the method different from last week’s implementation because it uses relative model behavior rather than only absolute signals from the two models.

2.2.2 Implementation

We implemented this method as a Python script in the `src` folder. For each sample, we extract features from both the 135M and 360M language models, including perplexity, loss, mean token probability, minimum token probability, maximum token probability, standard deviation of token probability, mean entropy, top-k mass, and low-confidence token ratio. We then concatenate the features from the two models and add delta features defined as

the difference between the corresponding values from the two models. These features are used to train a `GradientBoostingClassifier`. The model is trained on the training split, evaluated on the validation split using the official metrics, and then used to generate predictions for the test split in Kaggle submission format.

2.2.3 Results

Method	AUC	TPR@FPR = 0.1
Cross-Model Delta	0.8905	0.6496

Table 2: Validation performance of Model 2.

2.2.4 Discussion

This method works by capturing disagreement between a smaller and a larger language model on the same text. One advantage is that it provides more information since relative model behavior may reveal patterns that are not visible from one model alone. Another advantage is that the resulting features remain interpretable, because each delta feature corresponds to a measurable difference in model confidence or uncertainty. However, this method is a bit more computationally expensive than the baseline, since we created additional delta features. In addition, the current results are based on the validation split, so the final Kaggle performance may differ. This method could be improved in future work by combining cross-model deltas with stronger token-level features, or by applying feature selection to reduce redundancy and improve efficiency.

2.3 Model 3: Partial SPV-MIA with min-K% and XGBoost

2.3.1 Description

This model extends model 1 (Min-K% and XGBoost) outlined earlier in this paper. Model 3 added probabilistic variation features inspired by the SPV-MIA method outlined in the paper Fu et al. (NeurIPS 2024). The central idea is that memorized texts are positioned at a sharp peak in a model’s probability distribution. When comparing a memorized text to a slightly perturbed version, a larger drop in the model’s confidence in its probability prediction suggests that the text is likely a member text. In contrast, comparing non-memorized texts with their perturbed versions should result in similar model confidence, as the model is considering

both the original text and perturbed version equally, having memorized neither.

Slightly modified versions of each sample text are created using a T5 model to mask 30% of word spans in the original text, replacing them with similar alternative words. This maintains meaningful and context appropriate texts, despite having been modified. Loss is computed on the original and perturbed texts, which produced three new features: `spv_target`, `spv_reference`, and `spv_calibrated`. These features are combined with the existing features from Model 1.

The 135M and 360M models were used differently to generate these three new features. The 360M model was used as the "target" model and the 135M model was used as the "reference" model. SPV-MIA requires a target and a reference model as the probabilistic variation signal cannot decipher a probabilistic peak caused by memorization alone, versus one caused by the inherent complexity of the text. It is suggested that the reference model captures this natural complexity in the text, since it is laden with medical terminology, unusual words, and rare sentence structures. This provides a baseline from which we can subtract from the target model, which will have learned both the memorization and natural complexity signals. This subtraction equation leaves us with the memorization signal, the membership indicator.

$360M \text{ variation} - 135M \text{ variation} = (\text{memorization} + \text{natural complexity}) - (\text{natural complexity}) = \text{memorization signal}$

The 135M model was chosen as the reference, since it is smaller and less likely to have memorized text. Therefore, it should hold a stronger natural complexity signal. The 360M model was chosen as the target, as it is far larger and therefore more likely to contain memorizations alongside the natural complexity signal.

2.3.2 Implementation

Given the structure of the task, one feature of SPV-MIA, the self-prompting component where the target LLM is prompted to create a dataset, could not be implemented. Instead, the 135M pretrained and finetuned model was used as a substitute for the reference model.

The Model 3 pipeline was adapted from code provided by the authors of the paper Fu et al. (NeurIPS 2024). Several modifications were made in the course of implementing the model pipeline in order to complete computation within time con-

straints.

The T5 model was loaded to CPU, in an attempt to free GPU space for the 135M and 360M models. The T5 model output length was changed from 150 to 300 tokens to reduce reconstruction failures, which occurred when the T5 model could not create enough tokens to fill the masked spans required for each lengthy medical text. This aspect of the pipeline could certainly use some refining. Despite this change, reconstruction failures occurred at a rate of approximately 15-20% in each batch. When reconstruction failed, the original text was used instead, which means that the original loss and perturbed loss were both the same. For these examples, the SPV signal was 0.

Given time constraints, we also reduced the number of perturbations from 10 as outlined in the paper, to 2. We chose 2 in order to still provide the model with the ability to average the perturbation loss. To reduce duplicating calculations, the losses computed during feature extraction were cached so that they could be used again rather than recalculated during SPV feature calculation.

Despite all of the above modifications, computation remained quite slow at approximately 2 seconds per text, which forced us to compute SPV features on a very small subset of 3000 training samples, 500 validation samples and 500 test samples. The remaining samples received 0 values for SPV features.

The pipeline ran on the H100 GPU on Google Colab and GPU usage remained low, indicating that sequential computation time was the main issue, rather than memory usage. One option for future work would be to use `torch.compile()` when loading the SmolLM2 models, which compiles the model's operations once when they are loaded rather than requiring re-interpretation at each forward pass.

2.3.3 Results

Method	AUC	TPR@FPR=
Partial SPV-MIA with Min-K XGBoost	0.8955	0.6662

Table 3: Validation performance of Model 3.

Discussion

The SPV probabilistic features did not show improvement on milestone 2's Model 1 results, likely due to the necessary computational constraint of only calculating features on a small subset of 3000 training samples. Given that the remaining 94% of training samples had no SPV feature values, the

classifier likely learned to ignore SPV features entirely. Future attempts should access greater compute and calculate the SPV features on the entire dataset to accurately assess their usefulness.

3 Background

Summary of related research papers:

3.1 Paper 1: "What Hard Tokens Reveal: Exploiting Low-confidence Tokens for Membership Inference Attacks against Large Language Models" (Jawad et al., 2026)

This paper proposes an effective method for Membership Inference Attacks called "Hard Token Membership Inference Attack" (HT-MIA). Rather than concentrating efforts on the sequence of tokens in a given example, this method focuses on individual "hard tokens," which are low the probability tokens. The main contribution of this paper is that by focusing the low confidence token, instead of on sequences of tokens, the signal that the model trained on this data is much stronger. HT-MIA works with the following set of steps: Run both the fine-tuned target model and the original pre-trained reference model on a query sample. Then, identify the tokens where the target model has the lowest confidence. Next, compute the probability improvement at those positions, with improvement equal to the difference of the target minus the reference. Finally, score the sample by the fraction of hard tokens where the target model beats the reference. The models used by the writers of this paper are: GPT-2 (Radford et al., 2019), Qwen-3-0.6B (Yang et al., 2025), and LLaMA-3.2-1B (Dubey et al.2024). The datasets used in this paper are the following publicly available datasets: the Augmented Clinical Notes dataset (Bonnet et al., 2023), the Asclepius Synthetic Clinical Notes dataset (Kweonet al., 2023), the IMDB Large Movie Review dataset (Maas et al., 2011) and the Wikipedia text corpus (Wikipedia, 2022).

3.2 Paper 2: "Membership Inference Attacks on Large-Scale Models: A Survey" (Wu and Cao, 2025)

In "Membership Inference Attacks on Large-Scale Models: A Survey," Wu and Cao do not propose a new method for Membership Inference Attacks (MIA). Instead, they provide a comprehensive overview of MIA research in large-scale models. Since this is a survey paper, the authors do

not use any datasets or models. The main contribution this paper makes is to cover both Large Language Model (LLM) and Large Multimodal Models (LMM) model types and to categorize MIAs by pipeline stage, by adversarial knowledge and by attack strategy. The paper discusses the major attack families within LLMs: loss-based and perplexity-based methods, neighborhood/perturbation-based approaches, MIN-K% and MIN-K%++ , reference model methods, RAG systems and model alignment data. Going beyond LLMs, this paper advocates more work in LMMs, which the writers believe is an underexplored area. Overall, this paper describes the existing MIA methods, as well as suggesting MIA areas where more research is needed.

4 Bibliography

Michael Duan, Anshuman Suri, Niloofar Mireshghallah, Sewon Min, Weijia Shi, Luke Zettlemoyer, Yulia Tsvetkov, Yejin Choi, David Evans, and Hannaneh Hajishirzi. 2024. Do Membership Inference Attacks Work on Large Language Models? In *Proceedings of COLM 2024*.

Wenjie Fu, Huandong Wang, Chen Gao, Guanghua Liu, Yong Li, and Tao Jiang. 2024. Membership Inference Attacks against Fine-tuned Large Language Models via Self-prompt Calibration. *Advances in Neural Information Processing Systems*, 37:134981–135010.

Md Tasnim Jawad, Mingyan Xiao, and Yanzhao Wu. 2026. What Hard Tokens Reveal: Exploiting Low-Confidence Tokens for Membership Inference Attacks against Large Language Models. *arXiv preprint arXiv:2601.20885*.

Hengyu Wu and Yang Cao. 2025. Membership Inference Attacks on Large-Scale Models: A Survey. *arXiv preprint arXiv:2503.19338*.

Roy Xie, Junlin Wang, Ruomin Huang, Minxing Zhang, Rong Ge, Jian Pei, Neil Zhenqiang Gong, and Bhuwan Dhingra. 2024. ReCaLL: Membership Inference via Relative Conditional Log-Likelihoods. *arXiv preprint arXiv:2406.15968*.

5 Contributions

5.1 Milestone 2

Christina: Implemented model 1, posted Kaggle results, and created Overleaf document template. 0.27

Jennifer: Read articles and wrote summaries.
Did final document edits. 0.19

Rachelle: Implemented model 3 and posted Kaggle results. 0.27

Yucai: Implemented model 2 and posted Kaggle results. 0.27